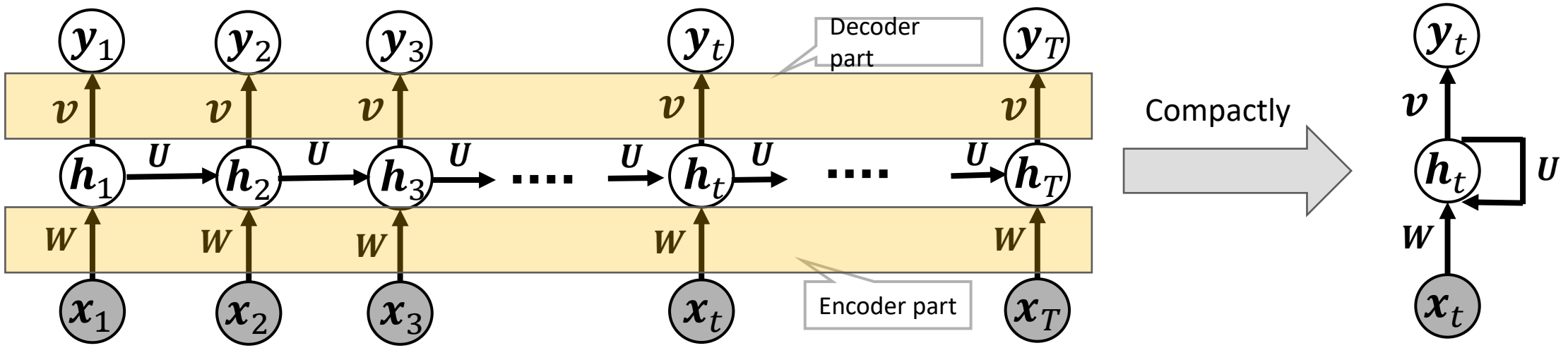
Autoregressive Model:
Attention Mechanism and Transformers

RNNs

- RNNs are used when each input or output or both are sequences of tokens

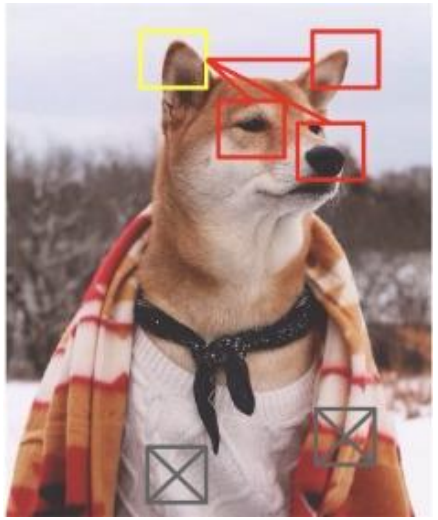


If the input is a word sequence then each x_n represent the corresponding word's embedding (either a pre-computed word embedding like word2vec or a learned word embedding)

- Hidden state $\boldsymbol{h}_t$ is supposed to remember everything up to time $t-1$. However, in practice, RNNs have difficulties remembering the distant past
  - Variants such as LSTM, GRU, etc mitigate this issue to some extent
- Slow processing is another major issue (e.g., can't compute $\boldsymbol{h}_t$ before computing $\boldsymbol{h}_{t-1}$)

Attention

- We use attention to “focus” on some part of interest in an input
  - Other nearby relevant parts help us focus
  - Other irrelevant parts do not contribute in the process





- In sequence modeling problems, we can use attention between input and output tokens (between encoder and decoder parts), as well as among the inputs only (only within the encoder part)

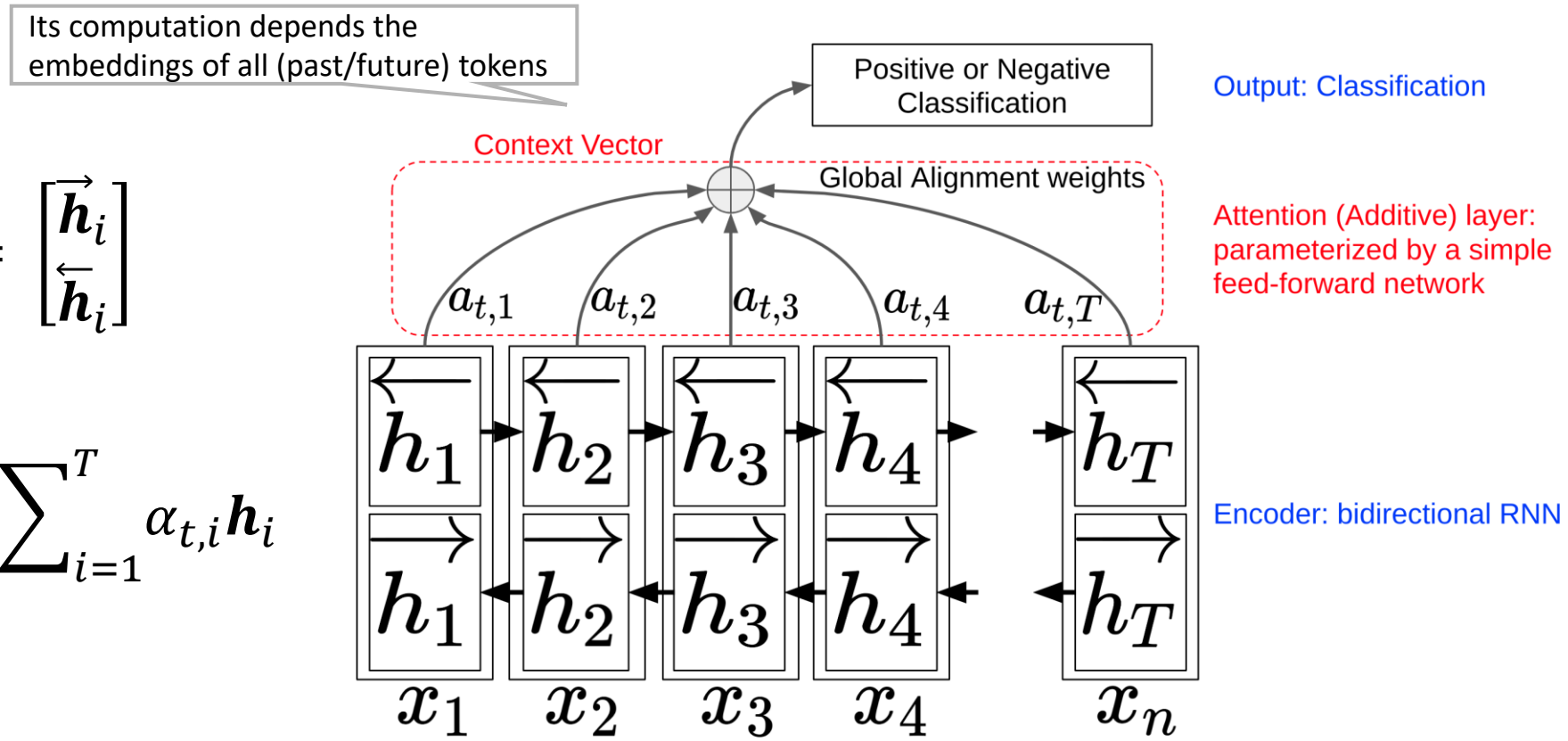
Pic credit: https://lilianweng.github.io/posts/2018-06-24-attention/

RNN with Attention

- RNNs have also been augmented with attention to help remember the distant past
- Attention mechanism for a bi-directional RNN encoder-decoder model

$$\boldsymbol{h}_i = \begin{bmatrix} \overrightarrow{\boldsymbol{h}_i} \\ \overleftarrow{\boldsymbol{h}_i} \end{bmatrix}$$

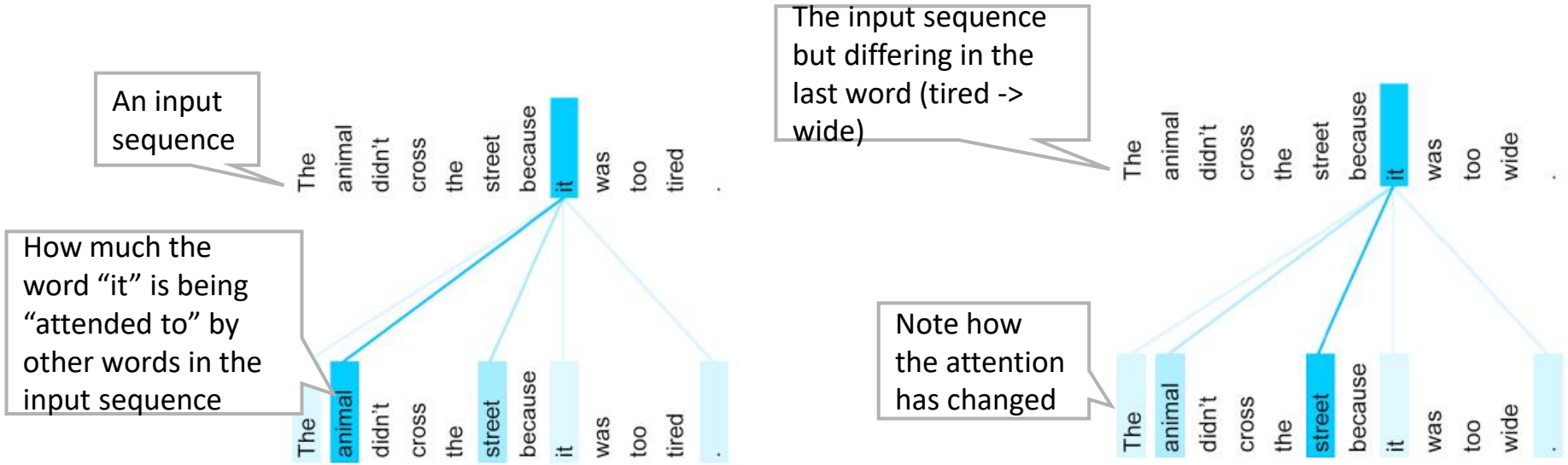
$$\boldsymbol{c}_t = \sum_{i=1}^{T} \alpha_{t,i} \boldsymbol{h}_i$$



Pic source: https://matthewmcateer.me/blog/getting-started-with-attention-for-classification/

Self-Attention

- With self-attention, each token $\boldsymbol{x}_n$ can "attend to" all other tokens of the same sequence when computing this token's embedding $\boldsymbol{h}_n$



- Attention helps capture the context better and in a much more "global" manner
  - "Global": Long ranges captures and in both directions (previous and ahead)

Example credit: https://blog.research.google/2017/08/transformer-novel-neural-network.html

Self-Attention

- For an N length sequence, the attention scores for each token $\boldsymbol{x}_n$ are computed using
  - A query vector $\boldsymbol{q}_n$ associated with that token
  - N key vectors $\boldsymbol{K} = \{\boldsymbol{k}_1, \boldsymbol{k}_2, \ldots, \boldsymbol{k}_N\}$ (one per token)
  - N value vectors $\boldsymbol{V} = \{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_N\}$ associated wit
- C compute the attention scor

$$\alpha_{n,i} = \frac{\exp(\boldsymbol{q}_n \boldsymbol{k}_i)}{\sum_{j=1}^{N} \exp(\boldsymbol{q}_n^\top \boldsymbol{k}_j)}$$

- Given attention scores, encoder's hidden state for $\boldsymbol{x}_n$ is

$$\boldsymbol{h}_n = \sum_{i=1}^{N} \alpha_{n,i} \boldsymbol{v}_i$$

$$\boldsymbol{H} = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}}\right)\boldsymbol{V}$$

Provided in form of a D-dim embedding (e.g., word2vec)

$N \times D$ matrix of original embeddings from the input layer

Linear projection by $D \times d$ matrix

Learnable

Row n is $\boldsymbol{q}_n$

$N \times d$ matrix of "queries"

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q$$

Assuming same size d as query

$N \times d$ matrix of N "keys"

$D \times d$ matrix. Learnable

$$\boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K$$

$N \times K$ matrix of "value" vectors of the N keys

$D \times K$ matrix. Learnable

$$\boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V$$

How much token i attends to token n

Dot-product attention (query and key assumed d dimensional)

$\boldsymbol{Q}$ and $\boldsymbol{K}$ are assumed $N \times d$

$N \times v$

$N \times v$

Attention-weighted sum of the value vectors of all the tokens in the sequence

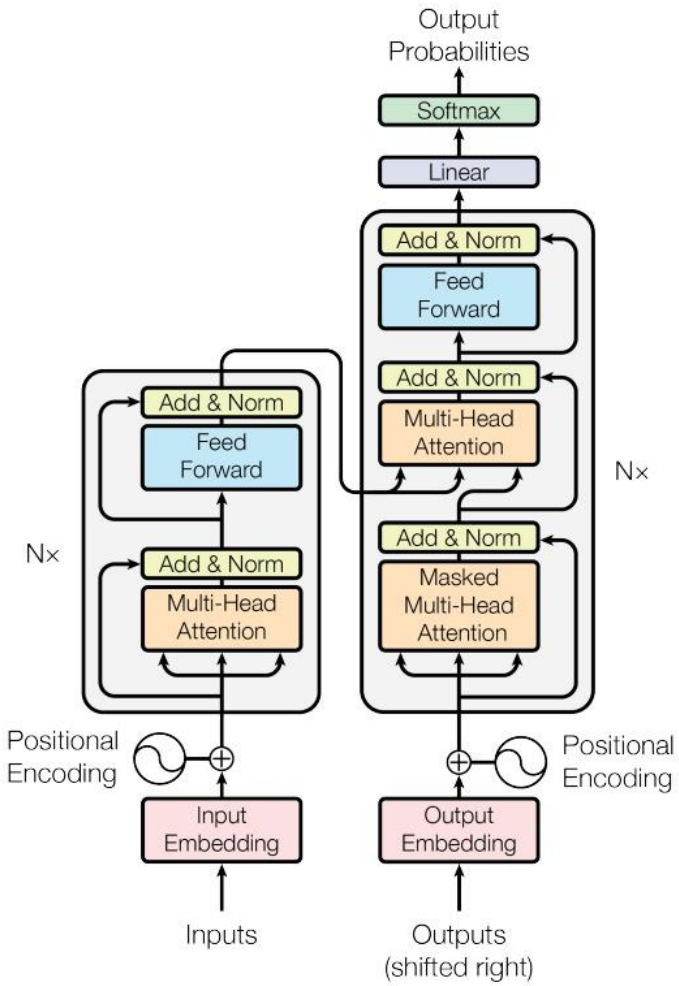
Thus the encoding of x_n depends on all the tokens in the sequence

Dividing by $\sqrt{d}$ ensures variance of the dot product is 1

"Scaled" dot-product attention

Transformers

- Transformers also use the idea of attention
  - “self-attention” over all input tokens
  - “self-attention” over each output token and previous tokens
  - “cross-attention” between output tokens and input tokens
- Transformer also compute embeddings of all tokens in parallel
- Transformers are based on the following key ideas*
  - “Self-attention” and “cross-attention” for computing the hidden states
  - Positional encoding
  - Residual connections
- Attention helps capture the context better and in a much more “global” manner in sequence data



“Attention is all you need” (Vaswani et al, 2017)

Positional Encoding

- Transformers also need a "positional encoding" for each token of the input since they don't process the tokens sequentially (unlike RNNs)
- Let $\boldsymbol{p}_i \in \mathbb{R}^d$ be the positional encoding for location i. One way to define it is

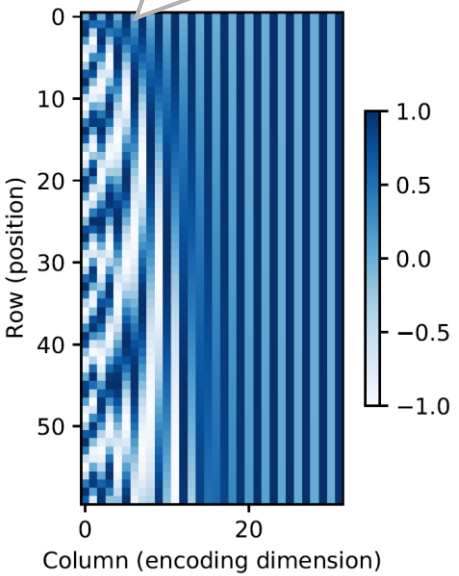
Here C denotes the maximum possible length of a sequence

$$p_{i,2j} = \sin\left(\frac{i}{C^{2j/d}}\right), \; p_{i,2j+1} = \cos\left(\frac{i}{C^{2j/d}}\right)$$

Positional encoding vector for location i assuming $d = 4$

$$\boldsymbol{p}_i = [\sin(\frac{i}{C^{0/4}}), \cos(\frac{i}{C^{0/4}}), \sin(\frac{i}{C^{2/4}}), \cos(\frac{i}{C^{2/4}})]$$

Note the smooth transition as the position index changes



- Given the positional encoding, we add them to the token embeddin

$$\hat{\boldsymbol{x}}_i = \boldsymbol{x}_i + \boldsymbol{p}_i$$

- The above positional encoding is pre-defined but can also be learned

Pic credit: PML-1 (Murphy, 2023)

Zooming into the encoder and the decoder..

FF operation is applied "position-wise" (for each token separately) but all FF blocks in the layer have same weights

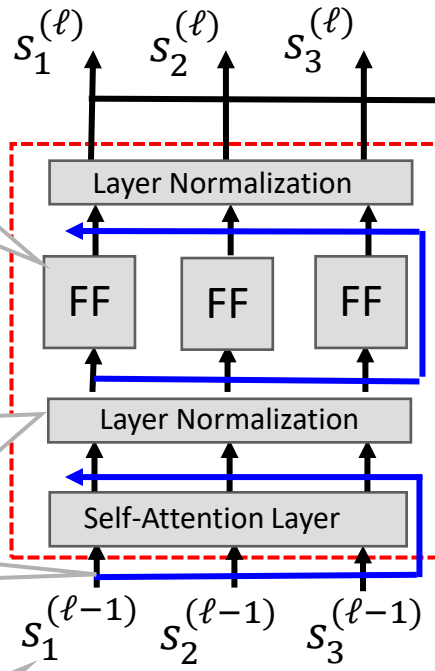
Each FF (feed-forward) is usually a linear layer + ReLU nonlinearity + another linear layer

Layer normalization (batch normalization is difficult since difference input sequences can be of different lengths)

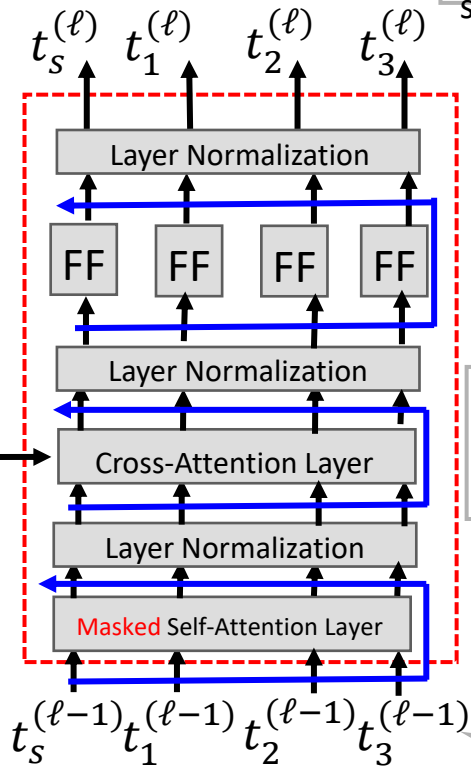
Blue arrows are the residual connections

Input ("source") token embeddings

Fixed in the first layer (obtained from an input embedding table) and learned for subsequent layers



An Encoder Block (N such blocks)



A Decoder Block connected with the corresponding encoder block (N such blocks)

Most likely output token at step m

"auto-regressive" generation

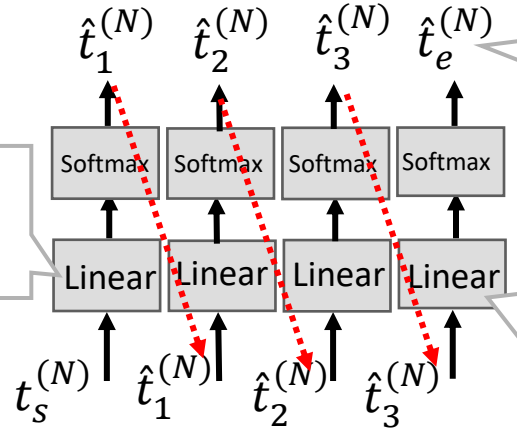
Output $\hat{t}_{m-1}^{(N)}$ from the previous step $(m-1)$ in the sequence

$$\hat{t}_m^{(N)} = \text{argmax}_{i=1,\ldots,V}\ \text{softmax}(\boldsymbol{W} t_m^{(N)})$$

t_e is a special end of sequence (EOS)token

Like FF, linear operation is also applied position-wise

With weight matrix W of size $V \times D$ where V is vocab size and D is the dimensionality of the last decoder block embeddings $t_m^{(N)}$



Decoder's output layer

Note the one position (towards right) shift between decoder's input vs output

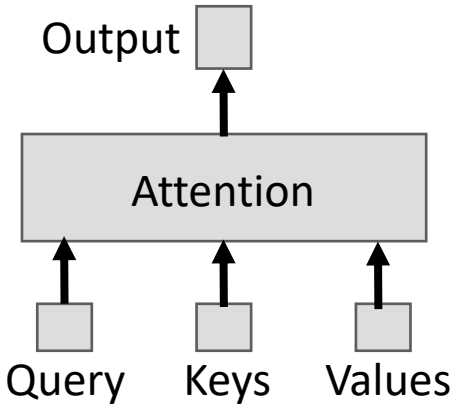
Output ("target") token embeddings

t_s is a special start of sequence (SOS)token

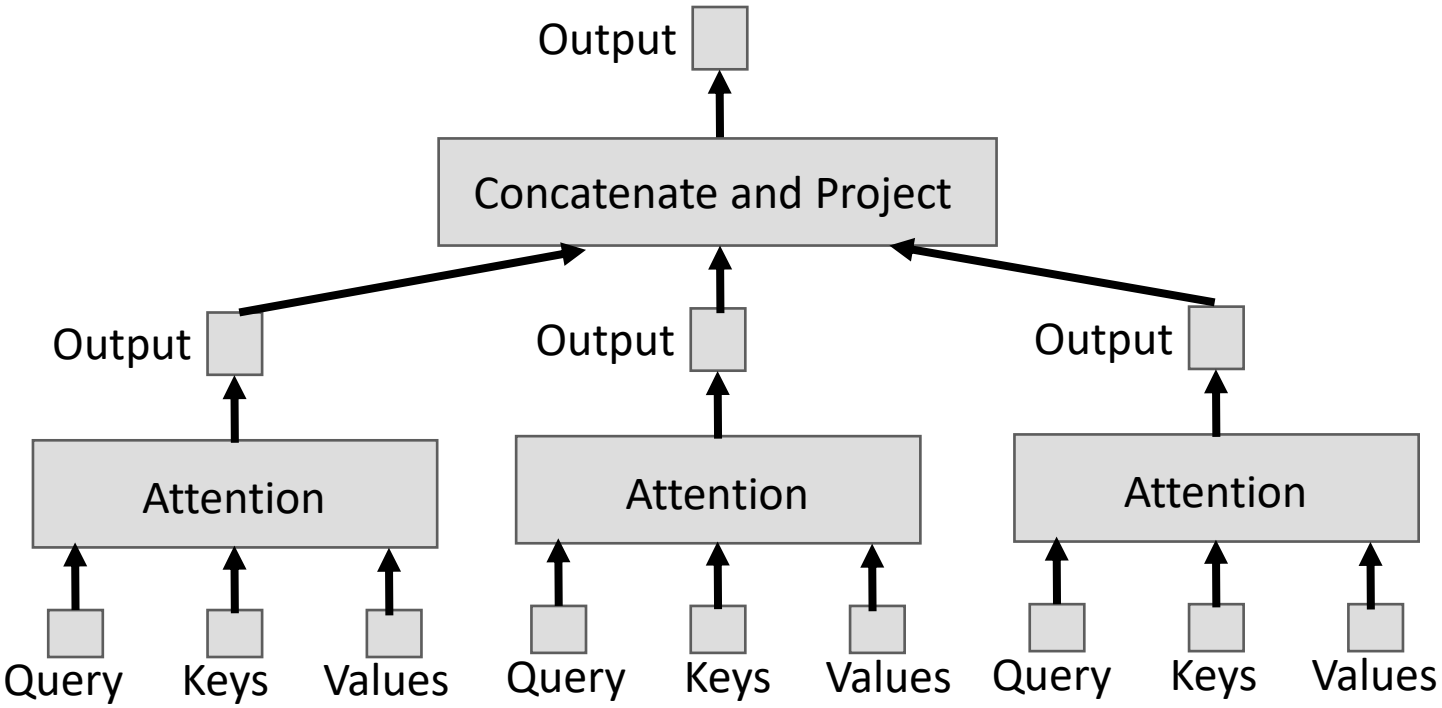
Multi-head Attention (MHA)

- A single attention function can capture only one notion of similarity
- Transformers therefore use multi-head attention (MHA)

Single Attention

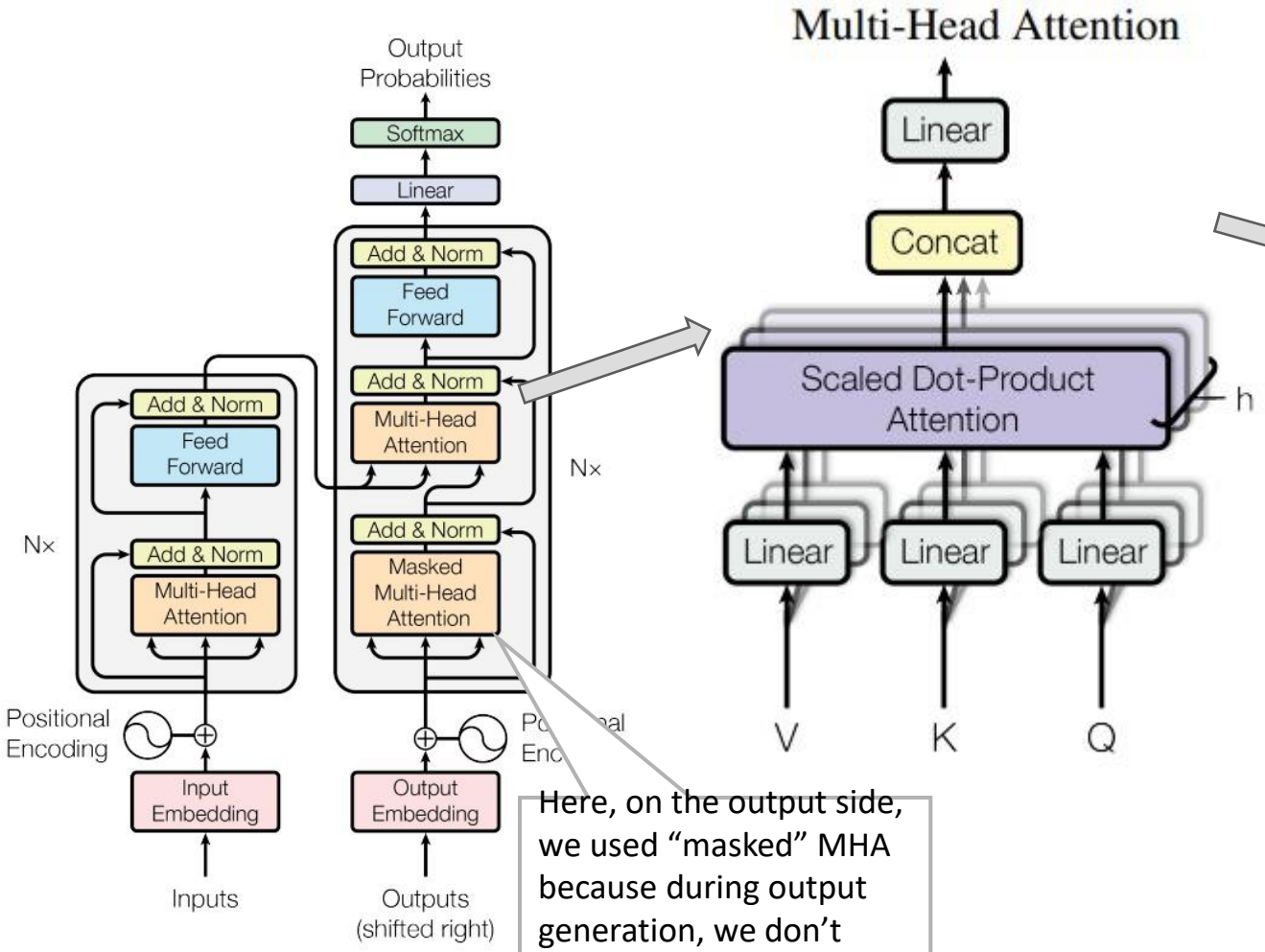


MHA

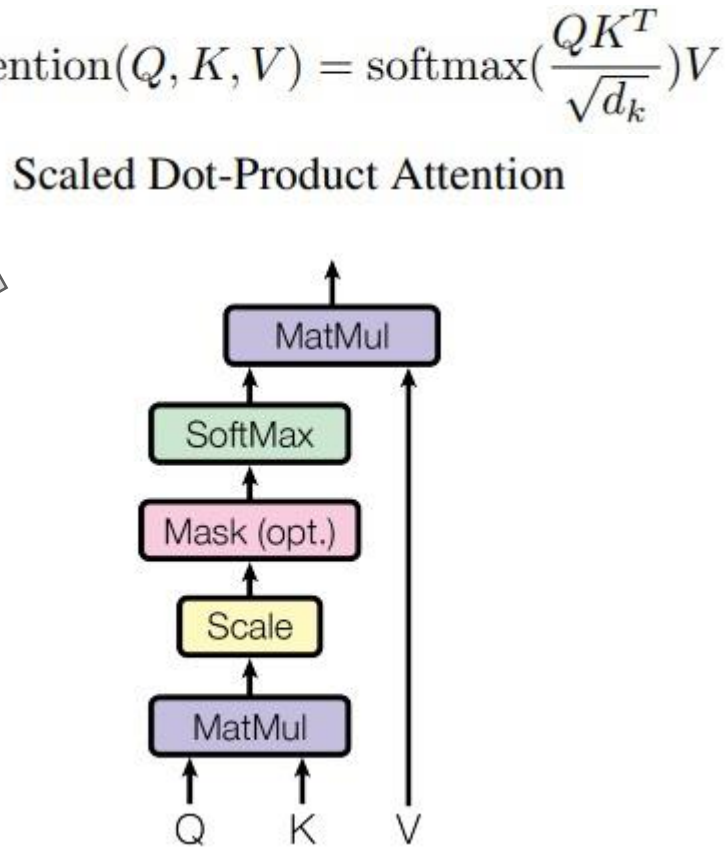


Pic credit: https://huggingface.co/learn/nlp-course/chapter1/

(Masked) Multi-head Attention (MHA)



$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



Here, on the output side, we used “masked” MHA because during output generation, we don’t want to look at future tokens

Pic source: “Attention is all you need” (Vaswani et al, 2017)

Computing Attention Efficiently

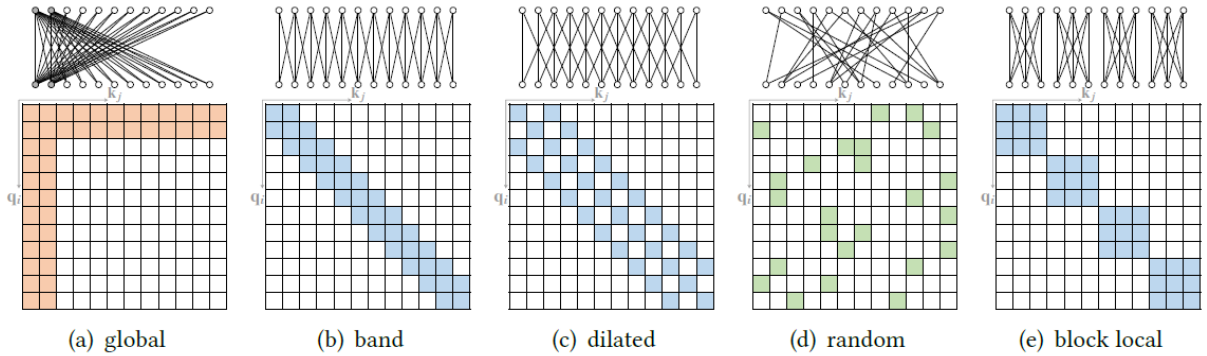
- The standard attention mechanism is inefficient for large sequences

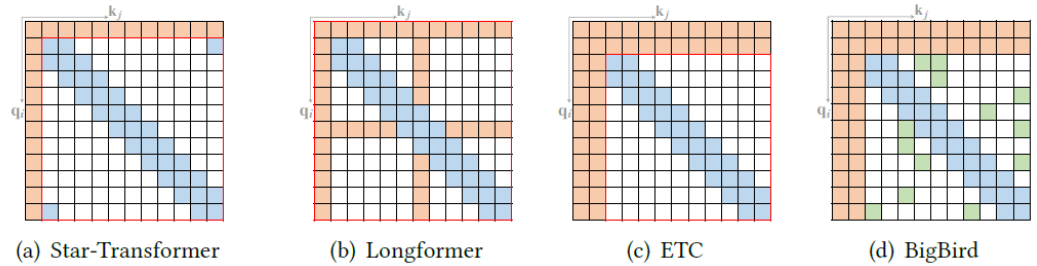
$O(T^2)$ storage and computation cost for a T length sequence

$$\boldsymbol{H} = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}}\right)\boldsymbol{V}$$

- Many ways to make it more efficient, e.g.,

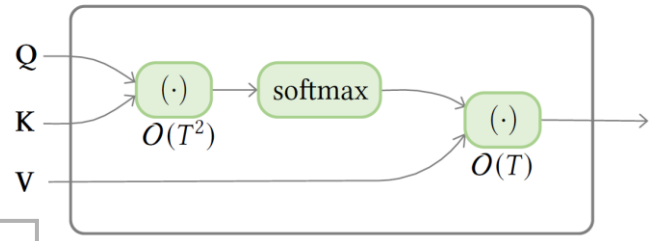
Sparse Attention





Linearized Attention

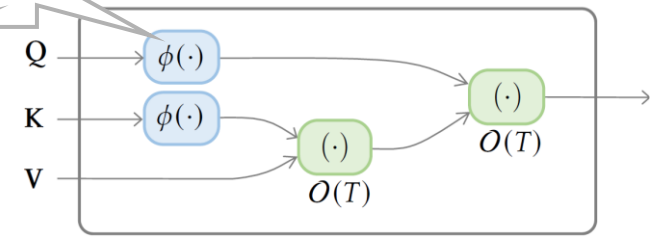
$$\exp(\boldsymbol{Q}\boldsymbol{K}^\top) \approx \phi(\boldsymbol{Q})^\top\phi(\boldsymbol{K})$$



(a) standard self-attention

A nonlinear projection based on kernels

E.g., kernel random features projection

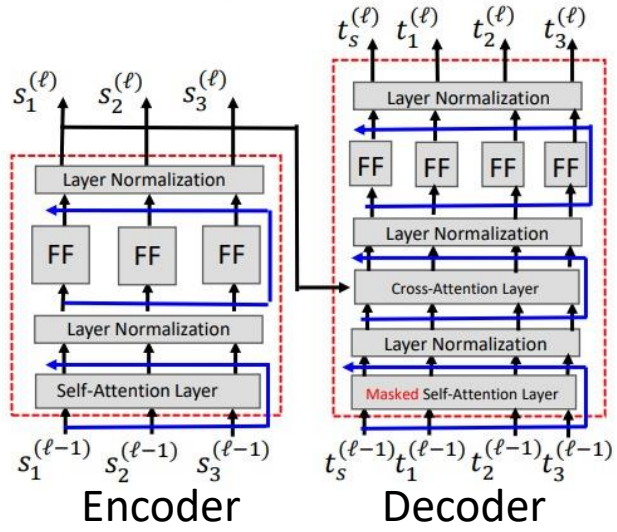


(b) linearized self-attention

Pic source: A Survey of Transformers (Lin et al, 2021)

Popular Transformer Variants: BERT and GPT

- The standard transformer architecture is an encoder-decoder model
- Some models use just the encoder or the decoder of the transforme
- BERT (Bidirectional Encoder Representations from Transformers)
  - Basic BERT can be learned to encoder token sequences
- GPT (Generative Pretrained Transformer)
  - Basic GPT can be used to generate token sequences similar to its training data

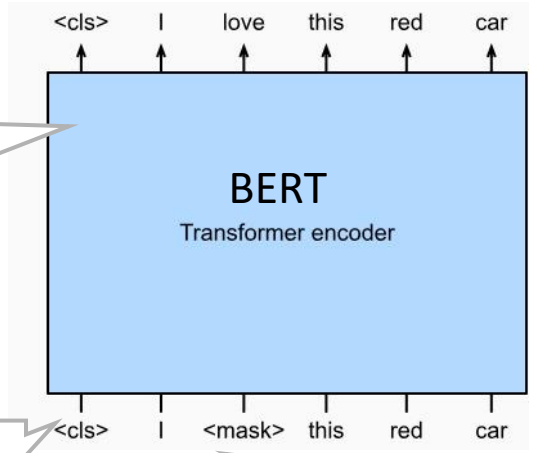


This encoder can be used for other tasks by fine-tuning

A transformer which contains only the encoder

Trained unsupervisedly using a missing token prediction objective

This is just start of sentence token

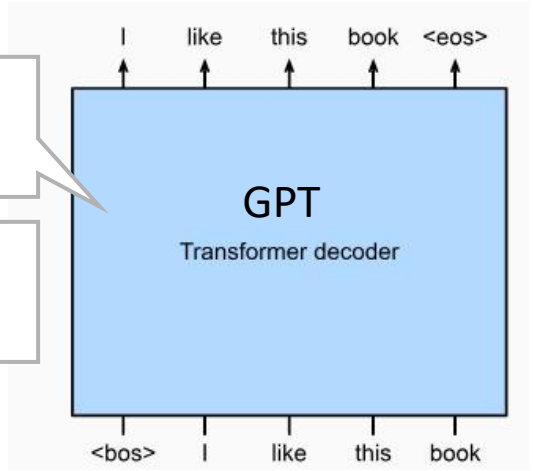


Missing token which BERT tries to predict

Also, no cross-attention since there is no encoder

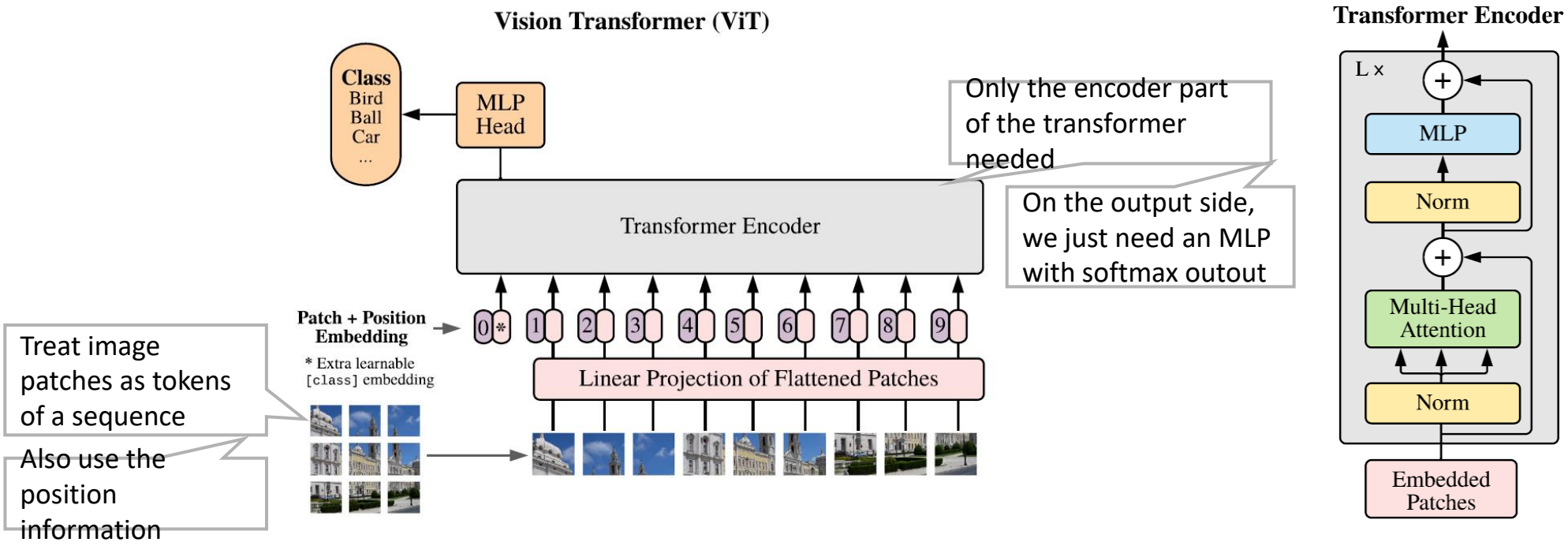
A transformer which contains only the decoder

Pre-trained using a next token prediction objective



Transformers for Images: ViT

- Transformers can be used for images as well#. For image classification, it looks like this



- Early work showed ViT can outperform CNNs given very large amount of training data
- However, recent work* has shown that good old CNNs still rule! ViT and CNN perform comparably at scale, i.e., when both given large amount of compute and training data

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (Dosovitskiy et al, 2020)

*ConvNets Match Vision Transformers at Scale (Smith et al, 2023)